COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S2
DATE PERFORMED:	07/24/2026	**DATE SUBMITTED:**	07/29/2026
NAME:	Juan Paulo C. Lara	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 3.1 Linked Lists

1. PROCEDURE OUTPUT

What procedures did you perform?

Screenshot

```cpp
#include <iostream>
#include <utility>

class Node{
public:
    char data;
    Node *next;
};

int main(){
    //step 1
    Node *head = NULL;
    Node *second = NULL;
    Node *third = NULL;
    Node *fourth = NULL;
    Node *fifth = NULL;
    Node *last = NULL;

    //step 2
    head = new Node;
    second = new Node;
    third = new Node;
    fourth = new Node;
    fifth = new Node;
    last = new Node;

    //step 3
    head->data = 'C';
    head->next = second;
    second->data = 'P';
    second->next = third;
    third->data = 'E';
    third->next = fourth;
    fourth->data = '0';
    fourth->next = fifth;
    fifth->data = '1';
    fifth->next = last;

    //step 4
    last->data = '0';
    last->next = nullptr;

    Node *temp = head;
    std::cout << "Linked List: ";
    while(temp != nullptr){
        std::cout << temp->data << "->";
        temp = temp->next;
    }
    std::cout << std::endl;

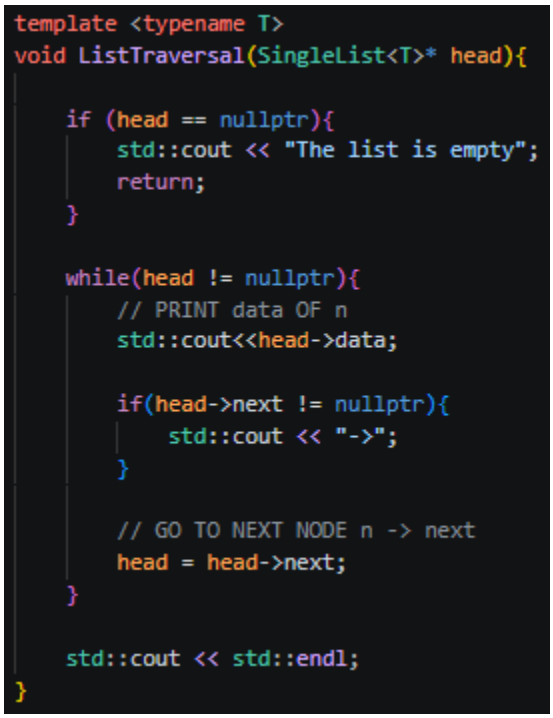
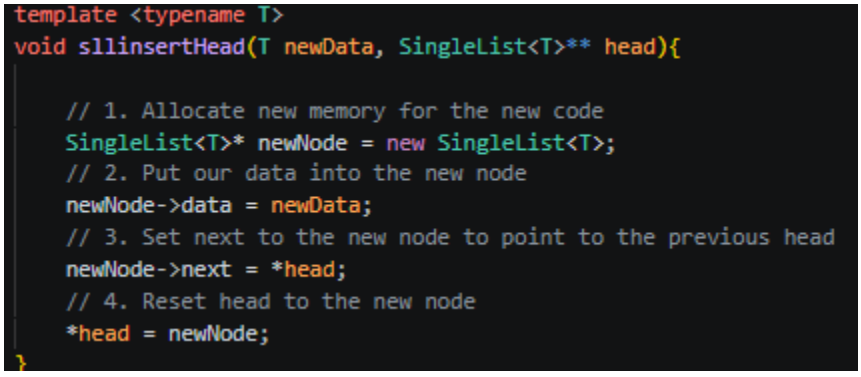
}
```

```
Linked List: C->P->E->0->1->0->
```

Discussion

This is the simple implementation of the linked list provided in the discussion, which does not use custom header files to perform these instructions. Instead, it uses the utilities library for basic linked list operations to initialize the node pointers as null, create new instances of the node class in the heap, define the node in the list by using char, and point the last node to nullptr. This program uses a simple linear way of declaring the chars in the linked list by performing the four steps stated before. The modification of this program is an addition of a block of instructions to display the elements of the linked list through the use of a while loop and a temporary variable. A temporary variable is used to store the value of the current head, and the while loop contains the condition that the temp is not a null pointer to execute the list traversal.

Table 3-1. Output of Initial/Simple Implementation

Operation	Screenshot
Traversal	
Insertion at head	

```
template <typename T>
void ListTraversal(SingleList<T>* head){

    if (head == nullptr){
        std::cout << "The list is empty";
        return;
    }

    while(head != nullptr){
        // PRINT data OF n
        std::cout<<head->data;

        if(head->next != nullptr){
            std::cout << "->";
        }

        // GO TO NEXT NODE n -> next
        head = head->next;
    }

    std::cout << std::endl;
}
```

```
template <typename T>
void sllinsertHead(T newData, SingleList<T>** head){

    // 1. Allocate new memory for the new code
    SingleList<T>* newNode = new SingleList<T>;
    // 2. Put our data into the new node
    newNode->data = newData;
    // 3. Set next to the new node to point to the previous head
    newNode->next = *head;
    // 4. Reset head to the new node
    *head = newNode;
}
```

Insertion at any part of the list

```
template <typename T>
void sllGeneralInsert(T newData, SingleList<T>* prevNode){

    //1. Check if it is the head node (previous node is null)
    if(prevNode == nullptr){
        //2. If null, print "Previous node cannot be null."
        std::cout<<"Previous value cannot be null"<<std::endl;
        return;
    }
    //3. Allocate a new node
    SingleList<T>* newNode = new SingleList<T>;
    //4. Store data in the new node
    newNode->data = newData;
    //5. Point new node to the node previous node was pointing to
    newNode->next = prevNode->next;
    //6. Point previous node to the new node
    prevNode->next = newNode;
}
```

Insertion at the end

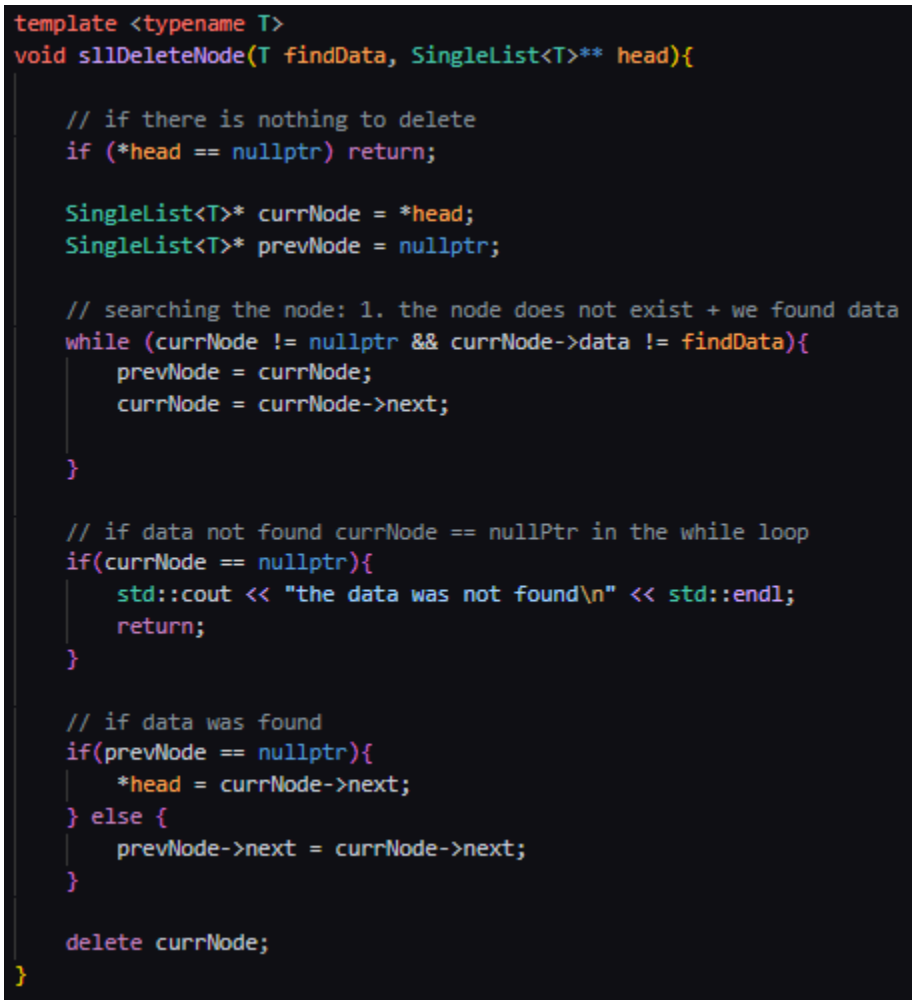
```
template <typename T>
void sllinsertEnd(T newData, SingleList<T>** head){
    // 1. Allocate a new node
    SingleList<T>* newNode = new SingleList<T>;

    // 3. Store data in new node
    newNode->data = newData;
    // 4. Point to null
    newNode->next = nullptr;

    // 2. Dereference the head node
    SingleList<T>* currentNode = *head;

    // 5. Traverse the list
    while(currentNode->next != nullptr){
        currentNode = currentNode->next;
    }

    // 6. Point the next of the current node to the new node
    currentNode->next = newNode;
}
```

Deletion of a node	

```
template <typename T>
void sllDeleteNode(T findData, SingleList<T>** head){

    // if there is nothing to delete
    if (*head == nullptr) return;

    SingleList<T>* currNode = *head;
    SingleList<T>* prevNode = nullptr;

    // searching the node: 1. the node does not exist + we found data
    while (currNode != nullptr && currNode->data != findData){
        prevNode = currNode;
        currNode = currNode->next;

    }

    // if data not found currNode == nullPtr in the while loop
    if(currNode == nullptr){
        std::cout << "the data was not found\n" << std::endl;
        return;
    }

    // if data was found
    if(prevNode == nullptr){
        *head = currNode->next;
    } else {
        prevNode->next = currNode->next;
    }

    delete currNode;
}
```

Table 3-2. Code for the List Operations

a Source Code:

```
7       DoubleList<char>* head = CreateNewNode('C');
8       DoubleList<char>* second = CreateNewNode('P');
9       DoubleList<char>* third = CreateNewNode('E');
10      DoubleList<char>* fourth = CreateNewNode('1');
11      DoubleList<char>* fifth = CreateNewNode('0');
12      DoubleList<char>* sixth = CreateNewNode('1');
13
14      head->next = second;
15      second->prev = head;
16
17      second->next = third;
18      third->prev = second;
19
20      third->next = fourth;
21      fourth->prev = third;
22
23      fourth->next = fifth;
24      fifth->prev = fourth;
25
26      fifth->next = sixth;
27      sixth->prev = fifth;
28
29      std::cout << "Original List: ";
30      dllTraverse(head);
```

Console:

```
Original List: Forward:
C P E 1 0 1
Backward:
1 0 1 E P C
```

b Source Code:

```
32      std::cout << "\nInsert G in the head:\n";
33      dllInsertHead('G', &head);
34      dllTraverse(head);
```

Console:

```
Insert G in the head:
Forward:
G C P E 1 0 1
Backward:
1 0 1 E P C G
```

c Source Code:

```
36      std::cout << "\nInsert E after P:\n";
37      dllGenInsert('E', second);
38      dllTraverse(head);
```

Console:

```
Insert E after P:
Forward:
G C P E E 1 0 1
Backward:
1 0 1 E E P C G
```

d	Source Code: Console:	40 std::cout << "\nDelete C:\n"; 41 dllDeleteNode(head->next, &head); 42 dllTraverse(head); Delete C: Forward: G P E E 1 0 1 Backward: 1 0 1 E E P G
e	Source Code: Console:	44 std::cout << "\nDelete P:\n"; 45 dllDeleteNode(second, &head); 46 dllTraverse(head); Delete P: Forward: G E E 1 0 1 Backward: 1 0 1 E E G
f	Source Code: Console:	48 std::cout << "\nNew List: "; 49 dllTraverse(head); New List: Forward: G E E 1 0 1 Backward: 1 0 1 E E G

Table 3-3. Code and Analysis for Singly/Doubly Linked Lists

Screenshots(s)	Analysis

```cpp
template <typename T>
void dllTraverse(DoubleList<T>* CurrentNode){
    // temp variable to store the last node visited
    DoubleList<T> *tail;

    // check if the list is empty
    if (CurrentNode == nullptr) {
        std::cout << "The list is empty" << std::endl;
        return;
    }

    std::cout << "Forward:\n";

    // Move forward and print
    while (CurrentNode != nullptr){
        std::cout << CurrentNode->data << " ";
        tail = CurrentNode;
        CurrentNode = CurrentNode->next;
    }

    // add a next line
    std::cout << std::endl;

    // move backward
    std::cout << "Backward:\n";

    while(tail != nullptr){
        std::cout << tail->data << " ";
        tail = tail->prev;
    }

}
```

This contains the traversal function implemented in a doubly linked list method. This uses a temp variable to store the last node accessed, and has an if statement to check if the list is empty. This traversal is done both forward and backwards for the output in the console.

```cpp
// Inserting at the head of the list
template <typename T>
void dllInsertHead(T newData, DoubleList<T> ** CurrentHead){

    // creating a new node
    DoubleList<T>* newNode = CreateNewNode(newData);

    // new node point to the current head
    newNode->next = *CurrentHead;

    // current head point back to newNode
    (*CurrentHead)->prev = newNode;

    // update the pointer head
    *CurrentHead = newNode;

}
```

This function dllInsertHead is used to insert a node at the head of the list by using the entered data from the function parameter. This creates a new node to create space for the new node in the list. Then, the new node points to the current head to point back to the node, and then, the pointer head is updated, so it can be accessed correctly by other functions.

```cpp
template <typename T>
void dllInsertEnd(T newData, DoubleList<T>* CurrentHead){
    // create a new node
    DoubleList<T>* newNode = CreateNewNode(newData);

    // traverse until we reach the last node
    while(CurrentHead->next != nullptr){
        CurrentHead = CurrentHead->next;
    }

    // connect the last node to the new node
    CurrentHead->next = newNode;

    // connect the new node to the last node
    newNode->prev = CurrentHead;

}
```

This dllInsertEnd function is where a node can be inserted at the end of the linked list. Like the previous functions, there is a new node created so it can be allocated. A list traversal is performed to head to the last node, so the last node can be connected to the last node.

```cpp
// GENERAL INSERT??
template <typename T>
void dllGenInsert(T newData, DoubleList<T>* CurrentNode){

    // create a new node
    DoubleList<T>* newNode = CreateNewNode(newData);

    // new node points to the next node
    newNode->next = CurrentNode->next;

    // new node points back to the current node
    newNode->prev = CurrentNode;

    // if current node is not the last node
    if(CurrentNode->next != nullptr){
        // next node points back to the new node
        CurrentNode->next->prev = newNode;
    }

    // current node points to the new node
    CurrentNode->next = newNode;

}
```

The general insert function called dllGenInsert is where a new node is inserted into the linked list. This creates a new node to point to the next node, and then the newly created node points back to the current node. Then, an if statement checks if the current node is not the last node or the null pointer. Inside it, the next node points back to the new node like before.

```
// DELETE A NODE??
template <typename T>
void dllDeleteNode(DoubleList<T>* DeleteNode, DoubleList<T>** CurrentHead){

    // check if the node is empty
    if(DeleteNode == nullptr){
        return;
    }

    // deleting the head node
    if(DeleteNode->prev == nullptr){
        *CurrentHead = DeleteNode->next;

        if(*CurrentHead != nullptr){
            (*CurrentHead)->prev = nullptr;
        }

    } else {
        // previous node points to the next node
        DeleteNode->prev->next = DeleteNode->next;

        // if the deleted node is not the last node
        if(DeleteNode->next != nullptr){
            // next node points back to the previous node
            DeleteNode->next->prev = DeleteNode->prev;
        }
    }
    // free the allocated memory
    delete DeleteNode;

}
```

This function called dllDeleteNode is where the deletion of the node is used to remove the node from a list and deallocate it from memory. The process first checks if the node is null or not. Another if statement is used to delete the head node and assign it to nullptr. Otherwise, the previous node heads to the next node and performs the deletion process. An if statement inside it checks if the next ones are not null, before it can point back to the previous node and delete the values inside. After this, the memory allocated by the nodes in the linked list is freed.

Table 3-4. Modified Operations for Doubly Linked Lists

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

```cpp
#include <iostream>
#include <string>
#include "circular_ll.h"

int main(){
    CircularList<std::string>* playlist = nullptr;
    CircularList<std::string>* current = nullptr;

    int choice;

    do{
        std::cout << "\n========================";
        std::cout << "\nSONG PLAYLIST";
        std::cout << "\n========================\n";
        std::cout << "1. Add Song\n";
        std::cout << "2. Remove Song\n";
        std::cout << "3. Play Playlist\n";
        std::cout << "4. Next Song\n";
        std::cout << "5. Previous Song\n";
        std::cout << "6. Exit\n";
        std::cout << "Enter Choice: ";
        std::cin >> choice;

        switch(choice){

        case 1:{
            char enterRepeat = 'y';
            std::string song;

            while(enterRepeat == 'y'){
                std::cout << "Enter song: ";
                std::cin.ignore();
                getline(std::cin, song);

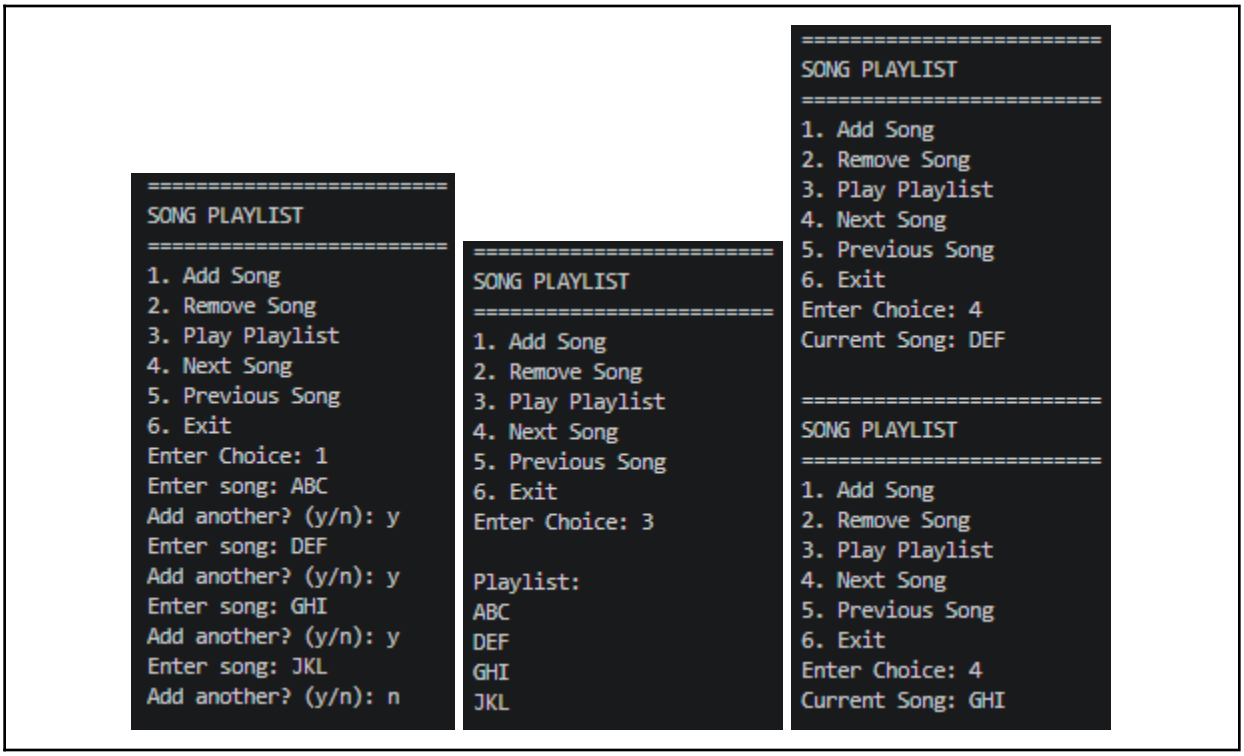
```

```
36                  addSong(song, &playlist);
37
38                  if(current == nullptr)
39                      current = playlist;
40
41                  std::cout << "Add another? (y/n): ";
42                  std::cin >> enterRepeat;
43              }
44
45              break;
46          }
47
48          case 2:{
49              std::string song;
50              std::cout << "Remove song: ";
51              std::cin.ignore();
52              getline(std::cin, song);
53              removeSong(song, &playlist);
54              break;
55          }
56
57          case 3:
58              playSongs(playlist);
59              break;
60
61          case 4:
62              nextSong(&current);
63              break;
64
65          case 5:
66              previousSong(&current);
67              break;
68
69          case 6:
70              deletePlaylist(&playlist);
71              std::cout << "Program End\n";
72              break;
73
74          default:
75              std::cout << "Invalid choice\n";
76          }
77
78      } while(choice != 6);
79
80      return 0;
81  }
```

Figure 1. Main Driver Code for the Playlist Program.

This code contains the main driver function to run the playlist program, set up the user input menu, declare the necessary variables and values, and set up the function calls from the header file. This driver code starts by

declaring the choice variable for the user input prompt. The program is mostly placed inside the do while loop block to continue asking the user for the choice until the user inputs a value to stop the program. When stopping the program, the playlist is also deleted and deallocated from memory, which will be shown in the header file in the next figures. Each case input contains the function invocations that will call the function definitions from the header file with the specified parameters from the playlist options, such as the current, playlist, song to be added to the circular linked list.



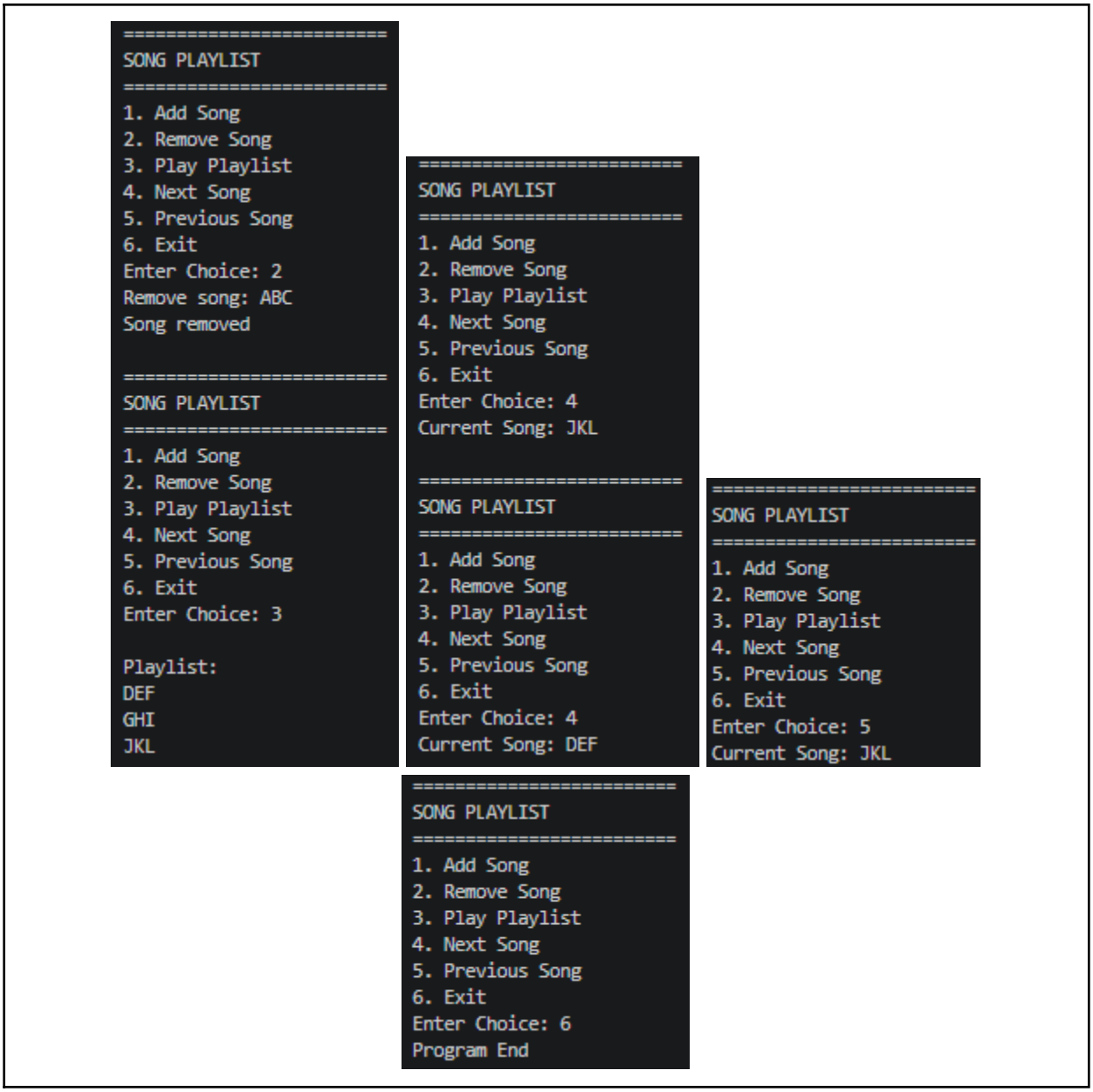


Figure 2. Console Output of the Playlist Program.

This figure contains the various console outputs of the playlist program from the driver code cpp file and the header file containing the functions and its definitions, and the templates based on the previously discussed singly and doubly linked list header files. The outputs demonstrate the options in the menu screen to show how the program functions in terms of how linked lists behave, and interact with the nodes inside the list. To keep it simple, three letter words were used instead to illustrate the sequence better. First, the song names were inputted sequentially based on the figure, which also uses a yes or no prompt to keep entering the song until the user stops. After this, the playlist of all the inputted songs are displayed. The circular linked list is tested with the previous and next options in the menu, wherein the song head DEF changes to GHI to show the next song, and a similar process is done for the previous option. Removing a song uses a user input string to remove the specified song, so it can be removed and deleted from the linked list. It is shown when ABC was removed from the playlist, and only the ones starting

from DEF remain.

```cpp
#ifndef CIRCULAR_LL_H
#define CIRCULAR_LL_H

#include <iostream>

template <typename T>
class CircularList{
    public:
        T data;
        CircularList<T>* next = nullptr;
        CircularList<T>* prev = nullptr;
};

template <typename T>
CircularList<T>* CreateNewNode(T data){

    CircularList<T>* node = new CircularList<T>;

    node->data = data;
    node->next = node;
    node->prev = node;

    return node;
}

```

```cpp
template <typename T>
void addSong(T data, CircularList<T>** head){

    CircularList<T>* node = CreateNewNode(data);

    if(*head == nullptr){
        *head = node;
        return;
    }

    CircularList<T>* tail = (*head)->prev;

    tail->next = node;
    node->prev = tail;
    node->next = *head;
    (*head)->prev = node;
}

```

```cpp
template <typename T>
void playSongs(CircularList<T>* head){

    if(head == nullptr){
        std::cout << "Playlist is empty\n";
        return;
    }

    CircularList<T>* temp = head;

    std::cout << "\nPlaylist:\n";

    do{
        std::cout << temp->data << std::endl;
        temp = temp->next;

    }while(temp != head);
}
```

```cpp
template <typename T>
void removeSong(T song, CircularList<T>** head){

    if(*head == nullptr)
        return;

    CircularList<T>* temp = *head;

    do{
        if(temp->data == song){

            if(temp->next == temp){
                delete temp;
                *head = nullptr;
                return;
            }

            temp->prev->next = temp->next;
            temp->next->prev = temp->prev;

            if(temp == *head)
                *head = temp->next;

            delete temp;
```

```
88                  std::cout << "Song removed\n";
89                  return;
90              }
91
92              temp = temp->next;
93
94          }while(temp != *head);
95
96          std::cout << "Song not found\n";
97      }
98
99      template <typename T>
100     void nextSong(CircularList<T>** current){
101
102         if(*current == nullptr)
103             return;
104
105         *current = (*current)->next;
106
107         std::cout << "Current Song: " << (*current)->data << std::endl;
```

```
111     template <typename T>
112     void previousSong(CircularList<T>** current){
113
114         if(*current == nullptr)
115             return;
116
117         *current = (*current)->prev;
118
119         std::cout << "Current Song: " << (*current)->data << std::endl;
120     }
121
122     template <typename T>
123     void deletePlaylist(CircularList<T>** head){
124
125         if(*head == nullptr)
126             return;
127
128         CircularList<T>* temp = (*head)->next;
129
130         while(temp != *head){
131
132             CircularList<T>* del = temp;
133             temp = temp->next;
134
135             delete del;
136         }
137
```

```
138
139         delete *head;
140         *head = nullptr;
141     }
142
143     #endif
```

Figure 3. Circular Linked List Header File based on the Singly and Doubly Linked List Header.

This is the header file and the main logic running this program before the driver code calls these functions. It contains the typical <typename T> templates for setting up the functions inside the header file, so it can be used in other cpp driver programs. This contains the process in creating, deleting, removing, and traversing through the linked list of nodes. The addSong function adds the input data from the driver code into the header file to add the song into the linked list. The playSongs function performs what is essentially a list traversal similar to the header files discussed before. The removeSong deletes the specified node inside the list, which is entered and set by the user in the driver code to call the header function to delete this part. The nextSong and previousSong are the functions that utilize the circular linked list method of traversing through the list in forwards and backwards respectively. The deletePlaylist deletes the allocated memory from the entered data in the linked lists and sets them to null.

Conclusion
This hands-on activity has demonstrated the use of singly, doubly, and circular linked lists in managing sequences of data through lists. This showed that there are alternatives to arrays through the linked lists demonstrated in this activity. It has three types which can be used to loop through a list, insert data, traverse a list forwards or backwards, enter new values in the middle of the linked lists, and many other ways in managing the sequence of nodes in the lists. The supplementary activities demonstrated the use of circular linked lists to allow a way to keep adding nodes to a list continuously, traverse through lists in a loop, and show the differences in the behavior of circular linked lists compared to its singly and doubly equivalents. Overall, this activity showed how to properly use linked lists, show their differences, and demonstrate the application of linked lists through a song playlist.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Google Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Summarization and clarification of concepts discussed**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **The AI was used to clarify the concepts of doubly linked lists**

COMPUTER ENGINEERING

QUEZON CITY

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.